Package 'mdi'

December 18, 2024

Type Package

Title Microdata Infrastructure: tools for statistical agencies and users at their research data centers

Version 0.2.1

License GPL-3

Author Eric Bartelsman, with contributions by Jing Chen, Mirja Halbig, Alessandro Zona Mattioli, Daniele Aglio, Marco Miorandi, Johanna Weiss, Reetuparna Vishwanath

Maintainer Comp-net Org <info@comp-net.org>

Imports data.table

Description The mdi package includes tools for statistical agencies (NSIs) and users at their research data centers. The tools are divided in four main categories:
- Infra: tools used to set up the NSI environment.
- MDI: general tools for NSIs and MDI users for purposes such as data aggregation, maniupulation etc.
- Analysis: research-related tools, used or statistical analysis.
- Programmer: additional tools including metadata handling, general hierarchical mappings etc.

Encoding UTF-8

LazyData true

RoxygenNote 7.3.2

Contents

concord . . . 2
concord_nace . . . 3
concord_nace_complex . . . 4
conc_tab . . . 5
conc_type . . . 7
consolidate_output . . . 8
disclosCrit . . . 8
disclose . . . 9
envir_prep . . . 10
estimate_markup . . . 11
estimate_outputelast . . . 12
estimate_prod . . . 13
gg_hist_density . . . 14
gg_scatter_reg . . . 15
harmonize_classification . . . 16

harmonize_variables 17
hier_apply 18
import_data 19
log_warnings 20
make_conc 20
make_filelist 21
mdi_aggregate 22
mdi_disagg 23
mdi_export 24
mdi_harmonize 25
mdi_import_mpnames 26
mdi_jointdist 26
mdi_mergedatatables 27
mdi_outlier 28
mdi_reclass 29
mdi_recode 30
mdi_redefine 31
mdi_regress 31
mdi_relpath 33
mdi_remap 33
mdi_revalue 34
mdi_sampleweights 35
md_cp2hier 35
pim_capital 36
read_firmdata_function 37

Index **39**

`concord` *NACE concordance*

Description

Tool for NACE concordance. The tool proceeds in 3 steps: Step 0: obtain conversion table with weights using function conc_tab() Step 1: apply target classification backwards for surviving firms Step 2: use concordance file for unambiguous (1:1 and m:1) matches STep 3: for ambiguous matches (1:m), randomly assign target classification according to realized switches of surviving firms

tooltype: Programmer

Usage

```
concord(
  DT,
  conc,
  id,
  time,
  ind,
  foryears,
  toyears,
  origin_class,
  target_class,
```

```
  weight_for = T,
  weight_type = "firms",
  impute_missing = TRUE
)
```

Arguments

DT	'data.table'; input dataset
conc	'data.table'; concordance file for industry classification
id	character; name of identifier variable in DT
time	character; name of time variable in DT
ind	character; name of industry variable in DT
foryears	vector; years of origin classification (for which to harmonize) -> the output dataset will only contain the years within 'foryears'
toyears	numeric; years of target classification (to which to harmonize)
origin_class	character; name of variable for original classification in conc
target_class	character; name of variable for target classification in conc
weight_for	boolean; forward method in computing weights (default as T)
weight_type	boolean; variable used to compute weights. Default uses number of firms. Alternatively, the number of employees ('emp') can be used
impute_missing	boolean; if TRUE, missing variables in ind are imputed

Value

It returns the input 'data.table, amended by the target classification and a flag whether the assignment was done in step 1, 2 or 3.

Note

Note: THIS TOOL ONLY RETURNS THE HARMONIZED INPUT 'data.table FOR A SELECTION OF YEARS PRIOR TO THE REFORM (to be specified in 'foryears')

Examples

```
concord(DT=br,conc=nace_rev11_rev2,id='firmid',time='year',ind='nace',foryears = c(2003:2007),
toyears=c(2008:2017),origin_class='nace_rev11_digit4',
target_class='nace_rev2_digit4')
```

---

concord_nace *NACE classification harmonization*

Description

This tool harmonizes NSI NACE classification over time. It simplify the harmonization process using the mode NACE code in the post break year as MD NACE code, and harmonizing accordingly previous year codes. It defines the break year as the year with the most changes in NACE codes.

Usage

```
concord_nace(dataset_name, weight_emp, cumsh_del, sh_del)
```

Arguments

dataset_name:	character to indicate the dataset name;
weight_emp:	logical to decide whether or not to weight code matches by employment (instead of number of firms).
cumsh_del:	number indicating the cumulative residual share of firms deleted for the construction of year by year concordances.
sh_del:	number indicating the share of firms deleted for the construction of year by year concordances.

Value

the original MD dataset with the old NSI NACE code and the standard MD NACE.

Author(s)

Daniele Aglio and Eric Bartelsman, 22/10/2024.

Examples

```
br_harmonized <- concord_nace_simple("br", FALSE, 0.95, 0.02)
```

---

concord_nace_complex *NACE classification harmonization*

Description

This tool harmonizes NSI NACE classification over time. It first harmonizes NSI NACE classification over time, based on firm-level data on the change in NACE code of firms, and it classifies NACE according to the last year version. Then, it maps NSI NACE classification to MD classification. It reads: the MD dataset (usually BR) with firmid, year, nace, persons_br. The structure of the tool is the following: firstly, the needed information about code concordances and paths is created from firm-level data; secondly, this information is use to harmonized NACE codes over years by firmid. In details, it creates 1) NSI NACE concordance tables from firm-level data for each couple of years; 2) history of NSI NACE codes from concordance tables; 3) concordances to last year NSI NACE codes; 4) it harmonizes NSI NACE codes over time, by firm: the resulting nace code is equal to the NSI code in the last year of the dataset; hence, previous years' codes are linked to the last one if the concordances allow for a unique link, otherwise they are randomly harmonized to the possible last year codes (weighted by firm-level evidence); 5) finally, it concords NSI NACE in the last year to MD nace. It also saves 1) in dirNSIMETA.

Usage

```
concord_nace_complex(dataset_name, weight_empl, cumsh_del, sh_del)
```

Arguments

dataset_name:	character to indicate the dataset name;
year_start:	number indicating the initial year of the dataset;
year_end:	number indicating the last year of the dataset;
weight_empl:	logical to decide whether or not to weight code matches by employment (instead of number of firms).
cumsh_del:	number indicating the cumulative residual share of firms deleted for the construction of year by year concordances.
sh_del:	number indicating the share of firms deleted for the construction of year by year concordances.

Value

the original MD dataset with the old NSI NACE code and the standard MD NACE.

Notes: matches between codes are defined as follows: 1 = 1:1 match; 2 = m:1; 3 = 1:m; 4 = m:m. In order to harmonize NACE codes, it is usually necessary BR, with the following variables: firmid, year, nace, persons_br. Byproducts: NSI NACE concordance tables for each couple of years; history of all NSI NACE code paths; NSI NACE concordance tables for each year to the last available year. Specific needed packages: data.table, igraph.

Author(s)

Daniele Aglio.

Examples

```
br_harmonized <- concord_nace("br", 2008, 2020, FALSE, 0.95, 0.02)
```

---

conc_tab *Concordance table building*

Description

Tool to build a concordance table using a dataset containing a column with an aggregation code and an official code conversion table. This returns a table with as many columns as in the official conversion table plus one column containing a weight measure. The weight measure can be calculated either in two ways. 1) foward, i.e. equal to the share of new codes observed out of the total of a specific old code; 2) backward, i.e. equal to the share of old codes observed out of the total of a specific new code.

tooltype Programmer

Usage

```
conc_tab(
  DT,
  conc,
  id,
  time,
  ind,
  foryears,
  toyears,
  origin_class,
  target_class,
  weight_for = TRUE,
  weight_type = "firms"
)
```

Arguments

DT	'data.table'; input dataset
conc	Input 'data.table'; concordance file for industry classification
id	character; name of identifier variable in DT
time	character; name of time variable in DT
ind	character; name of industry variable in DT
foryears	vector; years of origin classification (for which to harmonize)
toyears	numeric; years of target classification (to which to harmonize)
origin_class	character; name of variable for original classification in conc
target_class	character; name of variable for target classification in conc
weight_for	boolean; forward method in computing weights (default as T)
weight_type	boolean; variable used to compute weights. Default uses number of firms. Alternatively, the number of employees ('emp') can be used, as long as 'persons_br' is in the dataset

Value

Table with as many columns as in the official conversion table plus one column containing a weight measure.

Examples

```
conc_tab(DT=br,conc=conc, id='firmid',t='year',ind='nace',foryears = c(2003:2007),
toyears=c(2008:2017),
origin_class='nace_rev11_digit4',target_class='nace_rev2_digit4')
```

conc_type *Classify the concordance rows (edges)*

Description

This tool classifies concordance edges into type 1:1, 1:m, m:1, or m:m. Optionally, the tool also creates numbered 'components', or block-diagonals of the m:m edges. The input is a concordance data.table, with left and right columns and the function returns a data.table, possibly merged with the initial table, with conctype (and optional compnum).

Usage

```
conc_type(
  DTconc,
  left = "left",
  right = "right",
  mm_groups = TRUE,
  mergeconc = FALSE
)
```

Arguments

DTconc:	a concordance table (data.table) with two columns: given by parameters left and right.
left:	character ('left') is the name of the left side of the concordance
right:	character ('right') is the name of the right side of the concordance
mm_groups:	logical (TRUE) if m:m grouping (block diagonalization) is required.
mergeconc:	logical (FALSE) returns the conc merged with the original input

Value

Returns a data.table[‘left‘, ‘right‘,conctype, outdegree, indegree, compnum] where conctype gives type of match ("1:1","m:1","1:m","m:m"); outdegree gives number of right nodes to which a left node goes; indegree gives number of left nodes that go to a right node; if mm_groups=TRUE, then ‘compnum‘ gives component (block-diagonal) number. If mergeconc=TRUE then conc_type() returns the result merged with the original DTconc input.

Author(s)

Daniele Aglio and Eric Bartelsman, version Sep 22, 2024.

Examples

```
concx <- data.table(
  leftx = as.character(c(1, 2, 2, 3, 4, 5, 6, 6, 7, 8, 8)),
  rightx = c("A", "B", "C", "D", "D", "F", "E", "F", "H", "G", "H")
)
conc_type <- conc_type(concx, left = "leftx", right = "rightx", mm_groups = TRUE, mergeconc = TRUE)
```

consolidate_output *Stack Output Function*

Description

This function stacks output datasets based on the specified module name. It performs the following operations: - Reads a CSV file listing the outputs for each module, - Filters the list for the selected module, - Reads in the relevant files for each country where the file is available (indicated by CC=1/2/3) and appends data for different filenames, - Adds a country identifier to each dataset, - Stacks these datasets together.

This function assumes the existence of a CSV file that lists the outputs for each module. It requires the setup of the following directories: 'input_directory', 'output_directory', and 'metadata_directory'.

tooltype: Infra

Usage

```
consolidate_output(module_name)
```

Arguments

module_name	The abbreviation of the module name (e.g., EN, FD) as specified in the metadata file.

Value

An invisible data frame that combines all the stacked datasets. The function also saves this data frame as a CSV file in the 'output_directory' directory.

Author(s)

Reetuparna Vishwanath

Examples

```
consolidate_output("EN")
```

disclosCrit *Add disclosure criteria to the output table*

Description

The function 'disclosCrit' adds disclosure criteria to the output table. The function is called 'mdi_aggregate' if disclose == TRUE and adds to each aggregated dataset one column with the domVar (employment or sales) share of the top domNr firms per group (domSh) and one with the number of observations with non-missing employment or sales information (minNrObs)

tooltype: MDI

Usage

```
disclosCrit(DT, domVar = MDIdomVar, domNr = MDIdomNr, bygroups)
```

Arguments

DT	'data.table'; input dataset
domVar	character; variable based on which the dominance criterion is calculated; either employment ('emp') or sales ('nq)
domNr	numeric; number of top X firms for calculation of dominance criterion
bygroups	character; as specified in 'mdi_aggregate' function

Value

Data set with two columns added: one column with the domVar (employment or sales) share of the top domNr firms per group (domSh) and one with the number of observations with non-missing employment or sales information (minNrObs)

Author(s)

Marco Miorandi 15/10/2024 (last changes)

---

disclose *Disclosure check*

Description

The function 'disclose' conducts a disclosure check for aggregates based on the disclosure criteria that were added in 'mdi_aggregate'. The function is called in mdi_export for type 'sum_stats'. - Primary disclosure: replace all cells that do not satisfy dominance criterion or cells that include less than 'minNrObs' observations with '-999' - Secondary disclosure: replace smallest cell if only one cell has been surpressed in step 1 by '-999', only in case of totals/sums and if parent node is available tooltype: MDI

Usage

```
disclose(
  DT,
  hhfile = hhfile,
  hier = hier,
  minNumObs = MDIminNumObs,
  domSh = MDIdomSh
)
```

Arguments

DT	'data.table'; input dataset
hhfile	character; if used in hier_appl(): name of hierarchy file used for aggregation
hier	boolean; set to TRUE if there are multiple levels of aggregation (child and parent nodes) in the input dataset
minNumObs	numeric; minimum number of observations per cell required by disclosure, country-specific
domSh	numeric; maximum share of top X firms, country-specific

Value

The function returns a dataset with primary and secondary disclosures performed.

`envir_prep` *Environment preparation*

Description

Launch 1: This function 'envir_prep' is aimed at preparing the environement. This function takes as input the files MPnames_select and MPdata_select and provides the data in the target folder.

tooltype: Infra

Usage

```
envir_prep(
  vars_select = MPnames_select,
  years_select = MPdata_select,
  path = "H:/TMPSAVE/",
  panel_flag = T,
  defl = T,
  reset = F
)
```

Arguments

`vars_select`	.csv file indicating the needed variables across datasets. It is usually named MPnames_select
`years_select`	a.csv file indicating, for each desired dataset, how many years should be included. It is usually named MPdata_select
`path`	the target folder where the panels should be prepared
`panel_flag`	set TRUE if output files should be appended in path; default = T
`defl`	set TRUE if in path we should also include the deflators file
`reset`	set = T if the path should be emptied before running the function,

Value

The function stores in path, the final list of separate/appended datasets. If defl = T also a file called defl.csv will also be present.

Note

Function authored by Alessandro Zona Mattioli. Last update: 06/10/2022

Examples

```
#after you edit MPnames_select and MPdata_select, save them in .csv and run:
path <- 'H:/Microprod-H2020/MPproject/PilotTest/'
MPnames_select <- paste0(path,'MPnames_select.csv')
MPdata_select <- paste0(path,'MPdata_select.csv')
envir_prep(vars_select = MPnames_select,
           years_select = MPdata_select,
           path = 'H:/TMPSAVE/')
#if you want data to not be appended, run:
envir_prep(vars_select = MPnames_select,
          years_select = MPdata_select,
          path = 'H:/TMPSAVE/',
          panel_flag = F)
```

`estimate_markup` *Markup estimation*

Description

The function 'estimate_markup' estimates firm level markup based on De Loecker (2012) methodology.

tooltype: Analysis

Usage

```
estimate_markup(DT, oe = "oe_l", rev = "nq", input_cost = "nm")
```

Arguments

`DT`	input dataset
`oe`	output elasticity of the production function for a given input
`rev`	total revenues (turnover)
`input_cost`	total cost of the given input. For example, for labour use 'Personnel costs' (pay), for intermediate input 'Total purchases of goods and services' (nm) and for energy 'Purchases of energy products (in value)' (nnrg)

Value

The function returns a new variable with markup.

`estimate_outputelast` *Elasticity computation tool*

Description

This function 'estimate_outputelast' takes as input the regression output from a production function estimation and computes firm level elasticities. This computation changes according to the estimation method, where the possible methods are the same as in PilotProd: - OLS: implements a OLS residual with year FE and interactions with each input; - ACF: Ackerberg-Caves-Frazer (2015) methodology; - OP: Olley-Pakes (1996) methodology; - LP: Levinshon-Petrin (2003) methodology.

tooltype: Analysis

Usage

```
estimate_outputelast(
  DT,
  reg_output,
  l = "lnemp",
  k = "lnk_tan",
  m = "lnrm",
  years,
  method = "ols"
)
```

Arguments

`DT`	database where to add the new columns containing the elasticities
`reg_output`	regression output
`l`	input variable
`k`	input variable
`m`	input variable
`years`	list of years. This should be inputted in the same format as it would come out of unique(br_sbs$year) for instance
`method`	regression method

Value

The function returns one new variable per input, indicating elasticities: oe_k, oe_l, oe_m.

`estimate_prod` *Productivity Estimation Tool*

Description

This tool estimates productivity, allowing for multiple specifications of the production function and estimation methodologies. It can return input elasticities, TFP, or both, depending on the selected output_type. The function supports OLS, ACF, and OP methods, with translog (tl) or Cobb-Douglas (cd) specifications for the production function.

tooltype: Analysis

Usage

```
estimate_prod(
  DT,
  y,
  exog,
  endog = NULL,
  instr = NULL,
  fe = NULL,
  bygroups,
  id = NULL,
  time = NULL,
  method = "ols",
  spec = "cd",
  output_type = "ALL"
)
```

Arguments

`DT`	The input data.table containing all necessary variables for the estimation.
`y`	The output variable name. For ACF and OP methods, use value added.
`exog`	A vector of exogenous/state input variable names.
`endog`	Optional vector of endogenous/free input variable names.
`instr`	Optional vector of instrumental/proxy variable names.
`fe`	Optional vector of control/factor variable names for fixed effects.
`bygroups`	A vector of variables by which production functions are estimated separately.
`id`	Required for ACF and OP methods. Identifies the firm or entity.
`time`	Required for ACF and OP methods. Identifies the time period.
`method`	The estimation method: 'ols', 'acf', 'op'.
`spec`	The specification of the production function: 'tl' for translog or 'cd' for Cobb-Douglas. For OP, only 'cd' is allowed.
`output_type`	The desired output_type: 'el' for only elasticities, 'tfp' for TFP estimation only, or 'ALL' for both.

Value

A data.table column with the requested output: input elasticities and/or TFP, based on the output parameter selection.

Author(s)

Cindy Jing Chen, revised by Mirja Haelbig and further revised by Alessandro & Mirja

Examples

```
# OLS estimation for translog specification
cols <- c('el_k', 'el_l', 'el_m', 'tfp')
result <- estimate_prod(DT = br_sbs, y = 'lnrv', exog = c('lnk_tan', 'lnemp', 'lnrm'),
                  fe = 'year', bygroups = 'h_2', method = 'ols', spec = 'tl')

# ACF estimation for Cobb-Douglas specification
cols <- c('el_k', 'el_l', 'tfp')
result <- estimate_prod(DT = copy(db), y = 'lnrv', exog = 'lnk_tan', endog = 'lnemp', instr = 'lnrm',
                  fe = 'year', bygroups = 'h_2',
             id = 'firmid', time = 'year', method = 'acf', spec = 'cd', output_type = 'ALL')
```

---

gg_hist_density *Histograms*

Description

General tool for producting nice histograms.

tooltype: Analysis

Usage

```
gg_hist_density(
  DT,
  var_name = "nq",
  trim = NULL,
  both_tails = FALSE,
  density = TRUE,
  mean = TRUE,
  x_name = var_name,
  title = NULL
)
```

Arguments

DT	a ‘data.table‘ that contains the variable
var_name	variable name
trim	if NULL, no trimming, otherwise specify a pctile and the chart will plot only the observations below that pctile
both_tails	if TRUE, trimming is done also on the lower tail. Default is FALSE
density	fit a density over the histogram
mean	if TRUE, add a dashed line with mean to histogram
x_name	set name for x_axis. Default is var_name
title	set title of histogram. Default is ’Histogram of ’ + x_name

Value

It displays the resulting graph. It must be saved manually as saving options are not included.

`gg_scatter_reg` *(Analysis tool) Make x-y scatter plot and draw regression line.*

Description

'gg_scatter_reg()' displays the graph. Future updates with allow writing in in various formats.

Usage

```
gg_scatter_reg(
  DT,
  var_names,
  trim = NULL,
  both_tails = FALSE,
  reg_line = "lin",
  reg_weight = NULL,
  x_name = NULL,
  y_name = NULL,
  title = NULL
)
```

Arguments

`DT`	A data table containing the data.
`var_names`	character list (EB: is this right?) with the two (X,Y) variable names to compute the correlation/regression.
`trim`	integer if NULL, no trimming, otherwise specify a pctile and the chart will plot only the observations below that pctile. Default is NULL
`both_tails`	Boolean if TRUE, trimming is done also on the lower tail. Default is FALSE.
`reg_line`	character fit a regression line over the data cloud. Default is linear regression 'lin'. Other options include: - 'q' : quadratic regression; - 'c' : cubic regression; - 'poly' : 3rd degree polynomial expansion.
`reg_weight`	column specify weights for regression, it must be the name of a column of the data.table. Default is NULL.
`x_name`	character Set name for x_axis. Default is NULL
`y_name`	character Set name for y_axis. Default is NULL.
`title`	character Set title of histogram. Default is NULL

Details

tooltype: Analysis

Value

It displays the graph.

Examples

```
g_scatter_reg(myDT,c("yvar","xvyar"),trim=5,both_tails=TRUE,title="My Title")
```

`harmonize_classification` *Remap classifications*

Description

The function remaps the original 'data.table', including a target classification

Usage

```
harmonize_classification(
  DT,
  conc,
  native_class_DT,
  origin_class,
  target_class,
  rm_native_class_DT = F
)
```

Arguments

`DT`	'data.table' including the native_class_DT
`conc`	'character' name of the concordance table mapping native_class_DT to target_class
`native_class_DT`	'character' name of native classification in DT
`origin_class`	'character' name of native classification in concordance table
`target_class`	'character' name of target classification in concordance table

Details

tooltype: Infra

Value

The function returns a 'data.table' including the target classification

Note

Function authored by Mirja Haelbig. Last update: 11/05/2022

`harmonize_variables` *Remap NSI Variable Names to Microprod Variable Names*

Description

Remaps variable names in a dataset from NSI naming conventions to Microprod naming conventions and adjusts categorical values based on provided metadata. This process involves deleting variables not selected for analysis, renaming variables according to Microprod standards, and remapping categorical values using a provided codebook.

tooltype: Infra

Usage

```
harmonize_variables(DT, md_names_remap, md_names_select, md_codebook, ds, year)
```

Arguments

`DT`	Original ‘data.table‘ containing variables with NSI names.
`md_names_remap`	A metadata ‘data.table‘ that maps MP variable names to NSI variable names, including datasource and year-specific information.
`md_names_select`	A metadata ‘data.table‘ listing variables selected for inclusion in the analysis.
`md_codebook`	A metadata ‘data.table‘ containing concordances for remapping categorical values from NSI to Microprod standards.
`ds`	A string specifying the datasource prefix (e.g., "br" or "ofats").
`year`	Numeric specifying the year to which the data pertain.

Value

The input ‘data.table‘ (‘DT‘) after NSI variable names have been replaced with Microprod variable names and categorical values have been remapped.

Author(s)

Cindy Jing Chen authored on 20/08/2020 and revised by Mirja Haelbig on 01/03/2022 and Alessandro & Mirja on 23/03/2022.

Examples

```
# Assuming DT, md_names_remap, md_names_select, md_codebook are predefined:
remapped_DT <- harmonize_variables(DT, md_names_remap, md_names_select, md_codebook, "br", 2014)
```

`hier_apply` *Hierarchical Aggregation of Data*

Description

Aggregates variables in ‘var_list‘ to unique values of the hierarchical dimensions specified in ‘hhfile‘ by groups. This function allows for aggregation at various hierarchical levels specified by the user. The ‘mdi_aggregate‘ function is utilized for the aggregation process.

tooltype: MDI

Usage

```
hier_apply(
  DT,
  hhfile,
  var_list,
  bygroups,
  hier,
  agg_type = "sum",
  agg_weight = NULL,
  mrg = FALSE,
  disclosure = TRUE
)
```

Arguments

`DT`	The input dataset to be aggregated.
`hhfile`	A dataset containing columns of hierarchy; ‘bygroups[1]‘ must be included in this dataset.
`var_list`	A list of numeric variables in ‘DT‘ whose values are to be aggregated.
`bygroups`	A list of dimensional variables in ‘DT‘ to which the nodes are to be aggregated.
`hier`	Specifies the name of the aggregation hierarchy; if a single node (’h_x’), aggregates ‘h_0‘ to ‘h_x‘ through ‘hhfile‘. If ’ALL’, aggregates ‘h_0‘ to all available nodes.
`agg_type`	The type of aggregation required (options include "sum", "sd", "mean", "median", "aggweighted.mean", "count", "count.nna"); default is "sum".
`agg_weight`	List of variables used to ’weight’ the aggregation operation as defined by ‘agg_type‘; default is NULL.
`mrg`	Controls whether the results of the operation are merged into ‘DT‘; ‘FALSE‘ means ‘DTout‘ contains the result of the operation, ‘TRUE‘ means ‘DTout‘ is merged into ‘DT‘, and ‘DTout‘ is deleted.
`disclosure`	If ‘TRUE‘, dominance criteria and number of observations are added to the output dataset for disclosure control, applicable only if ‘mrg‘ is ‘FALSE‘.

Value

‘DTmg‘, the output dataset containing the aggregated dimensions and variables from ‘var_list‘.

Examples

```
hier_apply(DT, var_list, bygroups = c("nace", "year", "sizeclass"), indHier, hier="h_2", agg_type=c("mean","su
```

`import_data` *Import Data into R data.table*

Description

Reads data from various file formats into an R 'data.table'. This function is a wrapper around multiple file-reading packages such as 'fread' (from 'data.table'), 'haven' (for Stata, SAS, SPSS), and 'openxlsx' (for Excel). It supports CSV, Stata (.dta), Excel (.xlsx), SAS (.sas7bdat), SPSS (.sav), and tab-delimited text files (.txt). You can also specify a list of columns to import and specify which columns should be imported as characters. However, note that for some file formats (e.g., Stata, SAS, SPSS), the function cannot directly import columns as characters during the import process. In such cases, the specified columns are converted to character types **after** the data has been loaded.

Usage

```
import_data(dir, file, format, col_list = NULL, char_columns = NULL)
```

Arguments

`dir`	The directory path where the input file is located.
`file`	The name of the file to be imported.
`format`	The type of the file to be imported. Supported types include: - 'csv' for comma-delimited files (direct import as character supported), - 'txt' for tab-delimited text files (direct import as character supported), - 'dta' for Stata files (post-import conversion to character), - 'xlsx' for Excel files (post-import conversion to character), - 'sas7bdat' for SAS files (post-import conversion to character), - 'sav' for SPSS files (post-import conversion to character).
`col_list`	A character vector containing the names of the columns to import. If 'NULL' (default), all columns are imported.
`char_columns`	A character vector specifying which columns to treat as characters. For file formats where character conversion during import is not supported, the columns will be converted to character type **after** import.

Value

A 'data.table' containing the data from the imported file.

Note

For CSV and TXT files, the columns specified in 'char_columns' are imported as character during the data reading process. For Stata, Excel, SAS, and SPSS files, character columns are converted **after** the data is loaded into R.

Author(s)

EB, Marco Miorandi 14/10/2024

Examples

```
# Assuming 'dirINPUTDATA' is the directory and 'SBS_2014.csv' is the file:
data <- import_data("path/to/directory/", "SBS_2014.csv", "csv", char_columns = c("column1"))

# Importing specific columns from a Stata file and converting a column to character post-import:
data <- import_data("path/to/directory/", "datafile.dta", "dta", col_list = c("column1", "column2"), char_colu
```

`log_warnings` *Logging Warning to a txt File*

Description

This file is called by rocket.R and saves all warnings, diagnostics and additional information in a file "WarningDescription.txt" -> this is to be used instead of printing out to the console to information on the code

Usage

```
log_warnings(
  message,
  path = dirOUTPUT,
  append = TRUE,
  file_name = "WarningsDescription.txt"
)
```

Arguments

`message`	string for the message/warning that will be saved
`path`	outputdir of where file is stored, defaults to dirOUTPUT defined in the countdown
`append`	if to append to the file (as opposed to creating it)

`make_conc` *This tool creates concordance over the all relevant period for the classification of interest, starting from concordance tables. If required, it also group codes according to their match history. More precisely, this tool takes code concordance tables for each couple of years and creates the path (history) of each code from the first year to the last year of interest. Then, it groups paths that are linked by common codes, and to each group it assigns a common code equal to one of the last year codes in the group. This way, all codes are harmonized to a common set of codes, updated to the last year. Starting from a Babel tower, history gets harmonized again.*

Description

This tool creates concordance over the all relevant period for the classification of interest, starting from concordance tables. If required, it also group codes according to their match history. More precisely, this tool takes code concordance tables for each couple of years and creates the path (history) of each code from the first year to the last year of interest. Then, it groups paths that are linked by common codes, and to each group it assigns a common code equal to one of the last year codes in the group. This way, all codes are harmonized to a common set of codes, updated to the last year. Starting from a Babel tower, history gets harmonized again.

Usage

```
make_conc(conc_table, year_list, code_name, group_code)
```

Arguments

conc_table	concordance tables (data.table) for each couple of years;
year_list	years of interest (starting from the first year t in the first concordance table, not from t-1);
code_name	character name of the classification under analysis;
group_code	logical = TRUE if code grouping is required.

Value

data.table harmonized classification list in long shape;

reused_flag = 1 if some disappearing codes are reused;

data.table of grouped codes according to match, if group_code==TRUE.

Notes: the concordance tables must have code t-1 in the first column, code t in the second column.

Author(s)

Daniele Aglio.

Examples

```
conc_table_list <- make_conc(prodcom_conc_list, 2010:2021, "pcc8", TRUE)
```

---

make_filelist *Recursive File finder Find files (default: recursively) that fit a pattern*

Description

The function ‘make_filelist‘ returns a data.table with columns: name, extension, path from a list of full path filenames, found starting at the rootdir (recurseively searched if recursive=TRUE that match any of the strings given in the list patternlist tooltype: Programmer

Usage

```
make_filelist(rootdir, patternlist, recursive = T)
```

Arguments

rootdir	character; an absolute or relative directory to start the search for files
patternlist	list; a list of character strings with patterns. At least one must occur in the full file path/name
recursive	boolean: whether or not the search is recursive through the tree (default=TRUE)

Value

A dataset with matching files, split into name, extension and path

---

mdi_aggregate *Generic Aggregation Function*

Description

Performs aggregation from a sub-aggregate level to an aggregate level on a given dataset. Supports various aggregation types including sum, standard deviation, mean, median, count, number of missing values, number of zeros, number of positive values, and Herfindahl-Hirschman Index (HHI). Optionally, it can merge aggregated statistics as new variables into the input dataset and apply disclosure criteria.

tooltype: MDI

Usage

```
mdi_aggregate(
  DT,
  var_list,
  bygroups,
  agg_type = c("sum"),
  weight = NULL,
  mrg = FALSE,
  disclosure = TRUE
)
```

Arguments

DT	The original ‘data.table‘ to aggregate.
var_list	A vector of variables to aggregate.
bygroups	A vector specifying the levels of aggregation.
agg_type	A character vector specifying the type(s) of aggregation to perform. Supported types are "sum", "sd", "mean", "median", "count", "nmiss", "n_nonmiss", "nempty","nzero", "npos", and "HHI". Default is "sum".
weight	An optional variable for calculating weighted aggregates. Default is NULL.
mrg	A boolean indicating if the aggregated statistics should be merged back into the input dataset as new variables. Default is FALSE.
disclosure	A boolean indicating if dominance criteria and the number of observations should be included in the output dataset for disclosure control, applicable only if ‘mrg‘ is FALSE. Default is TRUE.

Value

An aggregated ‘data.table‘ containing the specified statistical measures.

Author(s)

Cindy Jing Chen, 21/08 2020; Additions by EB, 18/01/2024; Mod. df 28/05/2024, Marco Miorandi 15/10/2024

Examples

```
# Default usage
DTagg <- mdi_aggregate(DT, c('emp','rev'), c('nace','year'), 'sum')
# With weighted mean and count, including disclosure criteria
DTagg <- mdi_aggregate(DT, c('emp','rev'), c('nace','year'), c('mean','count'), 'popweight', FALSE, TRUE)
```

---

mdi_disagg *Assign Variable Values Across Hierarchical Dimensions*

Description

Assigns values from a category dimension in an input dataset (‘DT‘) to a more disaggregated category dimension, particularly useful when the dimensional variable in the input dataset encompasses multiple hierarchical levels. The function prioritizes assignment from the least aggregated dimension in ‘DT‘.

tooltype: MDI

Usage

```
mdi_disagg(DT, var_list, sourcecat, dim_o, target_cat, conc_cp)
```

Arguments

DT	The input dataset, which must contain the variables specified in ‘sourcecat‘ and ‘var_list‘.
var_list	A vector of numeric variables in ‘DT‘ whose values are to be disaggregated.
sourcecat	The dimensional variable in ‘DT‘ from which values are to be assigned.
dim_o	A vector of other dimensional variable names that, along with ‘sourcecat‘, uniquely identify records in ‘DT‘.
target_cat	The target category dimension used for disaggregation, which must be included in ‘conc_cp‘.
conc_cp	A data table containing the child-parent hierarchy for the disaggregation process.

Value

A data table (‘dbout‘) with values assigned from ‘sourcecat‘ to ‘target_cat‘ across the specified dimensions.

Examples

```
# Assuming DT, var_list, sourcecat, dim_o, target_cat, and conc_cp are predefined:
dbout <- mdi_disagg(DT, var_list, sourcecat, dim_o, target_cat, conc_cp)
```

mdi_export *Export Data and Log Output Description*

Description

Exports a given data table to a specified format (csv, RDS, txt, dta, xlsx, sas) and adds an entry to an output description file. This function is designed to facilitate the documentation and export of data outputs, including summary statistics, regression tables, or other types of outputs. It optionally supports handling hierarchical data structures for disclosure control.

tooltype: MDI

Usage

```
mdi_export(
  DT,
  hier = FALSE,
  hhfile = NULL,
  format,
  output_name,
  path = dirOUTPUT,
  desc_file = "OutputDescription",
  export_type = "sum_stat",
  description = " ",
  res_group = " "
)
```

Arguments

DT	A ‘data.table‘ object to be exported.
hier	Boolean flag indicating if there are multiple levels of aggregation in the dataset (TRUE for hierarchical data structures).
hhfile	Optionally, the name of the hierarchy file used for aggregation, relevant when ‘hier‘ is TRUE.
format	Character string specifying the format of the export (’csv’, ’RDS’, ’txt’, ’dta’, ’xlsx’, ’sas’).
output_name	The name of the file to be created, without the file extension.
path	The directory where the output file will be saved.
desc_file	The name of the file that logs descriptions of outputs, including the path and extension.
export_type	Character string indicating the type of output (’sum_stat’ for summary statistics, ’reg_tab’ for regression table, or ’other’).
description	A string providing additional explanation of the output file.
res_group	A string containing the abbreviation of the reserach group for which the export is done

Value

The function does not return a value but writes files to disk.

Examples

```
mdi_export(
  DT = datalist_num_firms[[nm]],
  hier = TRUE,
  hhfile = indHier,
  format = "csv",
  output_name = paste0(CountryCode, "Coverage_Number_Firms", nm, MPversion, sep="_"),
  path = dirOUTPUT,
  desc_file = "Output_description",
  export_type = 'sum_stat',
  description = 'Number of firms by datasource, industry (2 digit, 1 digit) and year'
)
```

`mdi_harmonize` *Harmonizes NSI data to MD dataset specifications*

Description

The function mdi_harmonize returns a datatable to MD specification, by changing the NSI variables using the metadata concordance file. The function stacks the 'body' of the functions: mdi_revalue mdi_recode mdi_reclass mdi_redefine mdi_remap see these functions for further details Examples: if a row of the concordance with method=revalue has 'detail' is: log(x), or x*3.14159, or format(as.date(x,"fmt1"),"fmt2") then this function executes DT[,NSI_varname:=log(NSI_varname)] or DT[,NSI_varname:=NSI_varname*3.14159] or DT[,NSI_varname:=format(as.date(NSI_varname,"

tooltype: Infra

Usage

```
mdi_harmonize(DT, concordance)
```

Arguments

`DT`	the name of the dataset whose variable values will be changed
`concordance`	is the name of the concordance dataset with rows containting NSI_varname, MD_varname, method, and detail

Value

executes all the changes using the detail for all the various harmonize methods

Author(s)

Eric Bartelsman 2024-05-29

mdi_import_mpnames *Cleaning MPnames*

Description

The function mdi_import_mpnames looks for all files starting with MPnames stored in dir, combines them and removes duplicates. MPnames should be csv files.

tooltype: Programmer

Usage

```
mdi_import_mpnames(dir)
```

Arguments

dir	directory path, the file path of the directory where the MPnames files are stored

Value

final_data, a dataset (data.table) which is a combination of all MPnames.csv files present in the directory (dir)

Author(s)

Leonardo Indraccolo, 03/2024

mdi_jointdist *Calculate Joint Distributions*

Description

Computes joint distributions for specified variables within a data table. This tool allows for the calculation of distributional moments (e.g., deciles, quintiles, quartiles) for one or more variables, aggregated by specified groups and potentially hierarchical structures.

tooltype: Analysis

Usage

```
mdi_jointdist(
  DT,
  hhfile,
  qnames,
  var_names,
  moment,
  bygroups,
  hier,
  agg_type,
  prefix = agg_type,
  agg_weight = NULL,
```

```
  mrg = FALSE,
  disclosure = TRUE
)
```

Arguments

DT	The input data table that includes variables for distribution calculations.
hhfile	A data table containing hierarchical information for aggregation.
qnames	A vector of names of variables used for calculating distributions.
var_names	The name of the variable(s) whose aggregates are computed.
moment	The distributional moment to compute for 'qnames'. Can be 'decile', 'quintile', or 'quartile'.
bygroups	Variables used for stratification in the distribution calculation.
hier	Specifies the hierarchical level for aggregation. 'ALL' indicates all levels; otherwise, specify the desired level.
agg_type	The type of aggregation to perform (e.g., sum, mean).
prefix	A prefix for naming aggregated variables, typically the same as 'agg_type'.
agg_weight	Optional; a variable name in 'DT' representing weights for aggregation.
mrg	A boolean indicating whether to merge results back with the original data table. Default is FALSE.
disclosure	A boolean indicating whether to apply disclosure control measures. Default is TRUE.

Value

A data table with the computed joint distributions, including the distributional moments for specified variables, aggregated according to the given criteria.

Examples

```
# Assuming DT is your dataset with variables 'income' and 'expenditure' and 'region' and 'sector' as bygroups
mdi_jointdist(DT, hhfile = hhData, qnames = c("income", "expenditure"), var_names = "income", moment = "quintil
```

---

mdi_mergedatatables *Merge Data Tables with Optional Variable Selection and Memory Management*

Description

Reads two data tables (left and right) and merges them according to specified keys, join type, and variable selection. Optionally 'pads' years to fill gaps in firms' time series if 'padyear' is set to TRUE. Provides an option to remove the original data tables from RAM to conserve memory.

tooltype: MDI

Usage

```
mdi_mergedatatables(
  DT_left,
  DT_right,
  vars_left,
  vars_right,
  mergekey = c("firmid", "year"),
  jointype = "inner",
  padyear = TRUE,
  rm_lr = FALSE
)
```

Arguments

DT_left	The left data table for merging.
DT_right	The right data table for merging.
vars_left	Variables from the left data table to keep in the merged dataset.
vars_right	Variables from the right data table to keep in the merged dataset.
mergekey	A vector specifying the keys for merging the data tables. Default is ‘c("firmid", "year")‘.
jointype	Type of join to perform. Can be "inner", "full", "left", or "right". Default is "inner".
padyear	Logical indicating whether to ’pad’ years in both data tables before merging. Useful for filling gaps in firms’ time series. Default is TRUE.
rm_lr	Logical indicating whether to remove the left and right data tables from RAM after merging. Default is FALSE.

Value

A merged data table based on the specified parameters.

Examples

```
# Assuming DT_left and DT_right are already defined and contain firmid and year variables.
mergedDT <- mdi_mergedatatables(DT_left, DT_right, vars_left = c("var1", "var2"), vars_right = c("var3", "var4
```

---

mdi_outlier *Outlier Routine for Data Cleaning*

Description

Executes a specified outlier handling routine (trimming, winsorizing, or flagging) on a dataset for selected continuous variables. This function can trim or winsorize the data at specified quantiles or flag observations as outliers based on the fraction provided. Trimming replaces outliers with NA, winsorizing replaces outliers with the closest value within the non-outlier range, and flagging marks outliers with a flag variable.

tooltype: MDI

Usage

```
mdi_outlier(DT, var_list, routine, fraction, both_tails = FALSE, group = NULL)
```

Arguments

DT	Input data table.
var_list	A vector of continuous variable names in 'DT' to be processed by the outlier routine.
routine	A string specifying the outlier routine to apply: "trim", "winsorize", or "flag".
fraction	The fraction of data to be trimmed or winsorized; must be a value between 0 and 1.
both_tails	Logical indicating whether to apply the routine to both tails of the distribution. If 'TRUE', the operation affects both the upper and lower tails; otherwise, it affects only the upper tail.
group	An optional variable name for grouping data before applying the outlier routine. Allows for outlier processing within specific groups.
flag	Logical indicating whether to flag observations as outliers instead of trimming or winsorizing them.

Value

A modified copy of 'DT' with the outlier routine applied to the specified variables. If flagging is selected, new columns are added to indicate flagged observations.

Examples

```
DT <- data.table(id = 1:10, income = c(50, 55, 45, 60, 200, 40, 45, 55, 65, 1000))
var_list <- c("income")
cleaned_data <- mdi_outlier(DT, var_list, "trim", 0.1, both_tails = TRUE)
```

---

mdi_reclass *Reclassify (concord) a variable Change the value of a DT classification variable, based on conc detail (_classconc) metadata*

Description

The function mdi_reclass concords the value of a class variable to values from another classification list The function reads the _classconc name from the detail of the concordance. It translates values from 'left' to 'right' side of the concordance. mdi_recode is not used, because tje concordance could be 'one-to-many'. In this case the algorithm will choose randomly from the 'right' side, but apply this to all records for a unique value of a variable (say firmid) Example: if DT has nace from NL_nace2008, then it chooses a NaceRev2 from NL_nace200_MD_nace_classconc.csv

tooltype: Infra

Usage

```
mdi_reclass(DT, conc)
```

Arguments

DT	the name of the dataset whose variable values will be changed
conc	is the name of the _conc concordance dataset that mentions the _classconc in the detail

Value

Dataset DT with the NSI_varname classification concorded to anoter classification

Author(s)

Eric Bartelsman 2024-05-24

Examples

```
DT <- data.table(firmid = rep(1:5, each = 2),
                 year = rep(2002:2003, times = 5),
                 nace = ifelse(rep(1:5, each = 2) == 5, 4, rep(1:4, each = 2)))
 DT[,nace1:=nace]
NSI_file1_vars_MD_filex_conc<-data.table(NSI_varname=c("nace","nace1"), MD_varname=c("nace2","nace3"), meth
nace_nace2_classconc<-data.table(left=c(1,2,3,4,4),right=c("A","B","B","C","D"))
newDT<-mdi_reclass(DT,NSI_file1_vars_MD_filex_conc)
```

---

mdi_recode — *Change the value of a DT categorical variable, based on _codeconc metadata*

Description

The function mdi_recode generates an R expression that changes the value of a variable in a dataset. The function makes use of R 'factors' and relabels their values Example: if a row of the concordance with method=recode has 'detail' is: NSI_ICTboolean_MD_ICTboolean_codeconc we essentially get DT[,NSI_varname:=factor(NSI_varname,levels=_codeconc$left, labels = _codeconc$right)] The program checks for unique NSI_varnames values not in the concordance and adds them to left and right

tooltype: Infra

Usage

```
mdi_recode(DT, conc)
```

Arguments

DT	the name of the dataset whose variable values will be changed
conc	is the name of the _codeconc concordance dataset mentioned in detail for 'recode' rows

Value

Dataset DT with the NSI_varname factor levels recoded

Author(s)

Eric Bartelsman 2024-05-24

`mdi_redefine` *Generate a 'right\ variable in DT based on additions and substractions of 'left' variables based on _conc metadata*

Description

The function mdi_redefine generates a new variable a arithmetic combinations of original variables The function reads a DT and a concordance and when method=redefine it searches all the 'left' rows for each unique 'right' row; It then sums all the left rows (with +/- from detail) to make the right variable in DT

tooltype: Infra

Usage

```
mdi_redefine(DT, conc)
```

Arguments

`DT`	the name of the dataset whose variable values will be changed
`conc`	is the name of the _conc concordance dataset

Value

Dataset DT with new RHS variable (with MDX_ prepended....)

Author(s)

Eric Bartelsman 2024-05-24

`mdi_regress` *Regression with Automatic Disclosure Check*

Description

Conducts regression analysis on the given data.table with an automatic disclosure check to ensure that the number of observations meets the minimum required threshold for disclosure. This function supports various regression models as specified in the 'fixest' package, including fixed effects OLS ('feols') and GLM ('feglm'). It can handle clustered standard errors and optionally export the regression results to a LaTeX file.

tooltype: Analysis

Usage

```
mdi_regress(
  DT,
  formula,
  model = "feols",
  family = NULL,
  vcov = "iid",
  cluster = FALSE,
  tex = F,
  output_name = NULL,
  desc_file = NULL
)
```

Arguments

DT	A ‘data.table‘ containing the dataset for regression analysis.
formula	A character vector of regression formulas.
model	The regression model to use, such as ’feols’ or ’feglm’. Default is ’feols’.
family	The family specification for GLM models, applicable if ’feglm’ is used. For example, ‘binomial‘.
vcov	The type of standard errors to be used. If using clustered standard errors, specify the clustering variable as ‘~clustervar‘. Default is ’iid’.
cluster	A boolean indicating if standard errors should be clustered. If ‘TRUE‘, ‘vcov‘ should be set to the name of the clustering variable. Default is ‘FALSE‘.
tex	A boolean indicating if the regression results should be exported as a LaTeX file. Default is ‘FALSE‘.
output_name	The name of the output file if ‘tex‘ is ‘TRUE‘. Default is ‘NULL‘.
desc_file	The name of the output description file if ‘tex‘ is ‘TRUE‘. Default is ‘NULL‘.

Value

A data.table containing the regression results if the disclosure criteria are met. Otherwise, a message is printed indicating that the regression does not satisfy the disclosure criteria.

Author(s)

Mirja Haelbig, 06/09 2022

Examples

```
DT <- data.table(export = rbinom(100, 1, 0.5), size = rnorm(100), firmid = rep(1:20, each = 5), year = rep(2010:2
mdi_regress(DT, formula = c('export ~ size', 'export ~ size | firmid + year'), model = 'feglm', family = binomial
```

mdi_relpath *Return the path relative to the root*

Description

Function used return a path relative to a root directory. This is used to take the path from NSI_datafiles and find the subdir relative to dirINPUTDATA If the root is not the beginning of the path, then the function returns the given path!

Usage

```
mdi_relpath(path, root)
```

Arguments

path	The string containing a windows or unix path name
root	The root of the directory containing given paths

Value

relative path

Author(s)

Eric Bartelsman 18-09-2024

Examples

```
NSI_datafiles[,path:=mdi_relpath(path,dirINPUTDATA)]
```

mdi_remap *mdi_remap*

Description

Renames variables of DT, using a conc with method=remap and renames NSI- to MD_varname.

tooltype: Infra

Usage

```
mdi_remap(DT, conc)
```

Arguments

DT	the name of the dataset whose variable names will be changed
conc	is the name of the _conc concordance dataset

Value

Dataset DT with variable names

Author(s)

Eric Bartelsman 2024-05-24

`mdi_revalue` *Change the value of a DT variable, based on metadata*

Description

The function mdi_revalue generates an R expression that changes the value of a variable in a dataset, by reference, using ‘detail‘ information in that row of the concordance file Example: if a row of the concordance with method=revalue has ’detail’ is: log(x), or x*3.14159, or format(as.date(x,"fmt1"),"fmt2") then this function executes DT[,NSI_varname:=log(NSI_varname)] or DT[,NSI_varname:=NSI_varname*3.14159] or DT[,NSI_varname:=format(as.date(NSI_varname,"

tooltype: Infra

Usage

```
mdi_revalue(DT, concordance)
```

Arguments

`DT`	the name of the dataset whose variable values will be changed
`conc`	is the name of the concordance dataset with ’revalue’ rows
	example DT[,NSI_varname:= function_of_x_in_detail]
	DT <- data.table(firmid = rep(1:5, each = 2), year = rep(2002:2003, times = 5), radius = rep(1:5, each = 2))
	NSI_file1_vars_MD_filex_conc<-data.table(NSI_varname=c("radius"), MD_varname=c(""), method=c("revalue"), detail=c("x*3.14159*2"))
	newDT<-mdi_revalue(DT,NSI_file1_vars_MD_filex_conc)

Value

executed calculation, by reference, to the NSI_varname in the dataset DT

Author(s)

Eric Bartelsman 2024-05-24

mdi_sampleweights *Calculate Sample Weights*

Description

Calculates sample weights based on the provided parameters. This function: - Excludes rows with missing values in specified variables. - Computes sample weights by dividing the theoretical number of firms by the number of observations in each stratification group. - Ensures that no sample weight is less than one. - This process is aligned with sample reweighting techniques as described in ProdSys documentation.

tooltype: Analysis

Usage

```
mdi_sampleweights(DT, var_list, popweight, bygroups)
```

Arguments

DT	A data table containing the data.
var_list	A vector of variable names in 'DT' that should not have missing values.
popweight	The theoretical number of firms, used as a numerator for calculating weights.
bygroups	Variables that define stratification groups for calculating weights.

Value

A modified copy of 'DT' including a new column 'sampleWeight' that contains the calculated sample weights for each observation.

Examples

```
DT <- data.table(firm = 1:100, industry = rep(1:10, each = 10), sales = runif(100))
var_list <- c("sales")
popweight <- 1000
bygroups <- "industry"
DT_weighted <- mdi_sampleweights(DT, var_list, popweight, bygroups)
```

md_cp2hier *Hierarchical Mapping from Child to Parent Nodes*

Description

Performs hierarchical mapping from a specified child node to its parent nodes in a given dataset. This function is useful for hierarchical data structures where each child node may be associated with one or more parent nodes at specified hierarchical levels.

tooltype: MDI

Usage

```
md_cp2hier(DT, hhfile, hlist_c, hlevel)
```

Arguments

DT	A ‘data.table‘ containing the nodes (e.g., ‘h_0‘, ‘h_1‘, ‘h_2‘, ...) and corresponding category codes (e.g., NACE codes).
hhfile	A ‘data.table‘ representing the metadata with columns of hierarchy that map child nodes to parent nodes.
hlist_c	The name of the child node column in ‘DT‘ that will be mapped to its parent nodes. Must be one of the columns in ‘hhfile‘.
hlevel	An integer vector specifying the level(s) of the parent node(s) relative to the child node. For example, ‘1L‘ for the immediate parent, ‘2L‘ for the grandparent, etc. Can specify multiple levels to retrieve multiple layers of parentage.

Value

A list containing two elements: ‘DTout‘, the ‘data.table‘ augmented with the corresponding parent node(s), and ‘hlist_p‘, the name(s) of the parent node column(s).

Examples

```
# Assuming DT, hhfile, and indHier are predefined and contain appropriate hierarchical structure:
result <- md_cp2hier(DT, hhfile, "h_1", c(1L, 2L))
```

---

pim_capital *Estimate Capital Stock Using Perpetual Inventory Method (PIM)*

Description

Estimates capital stock based on the Perpetual Inventory Method (PIM), as outlined in Halle & Mairesse (1995). This function is tailored for data structured as panel data, with indexing based on a firm identifier (‘firmid‘) and a year variable. It involves the following steps: - Reading the specified variables from the input data.table. - Depending on whether the depreciation rate is provided directly or inferred from the asset type, the function calculates the capital stock. - The output is the input data.table augmented with the new capital stock variable.

The function supports different depreciation formats and asset types, offering flexibility in estimating capital stock across various contexts.

tooltype: Analysis

Usage

```
pim_capital(
  DT,
  I = "ni_eq",
  depr = "dep_ME",
  asset_type = NULL,
  G = "g_ME",
  dep_format = "pct",
  output_name = "k_new"
)
```

Arguments

DT	A data.table including the necessary variables.
I	The column name for real investment. Default is "ni_eq".
depr	The column name for the depreciation rate. If 'NULL', an asset type must be specified in 'asset_type'. Default is "dep_ME".
asset_type	Specifies the asset type if 'depr' is 'NULL'. Valid options are 'ME', 'CO', 'Int', 'SW', 'RD'. Default is 'NULL'.
G	The column name for the geometric mean of annual growth rates of investment at the industry level. Default is "g_ME".
dep_format	Specifies the format of the depreciation variable. If 'pct', the depreciation variable is assumed to be a percentage. If 'num', it is assumed to be numeric. The formula is adjusted accordingly. Default is 'pct'.
output_name	The name of the output variable for the estimated capital stock. Default is "k_new".

Value

A modified 'DT' that includes the new capital stock variable.

Author(s)

Version 1.1, 28 June 2021

References

Halle, P., & Mairesse, J. (1995). "Estimation of the Perpetual Inventory Method for Capital Stock".

Examples

```
DT <- data.table(firmid = 1:5, year = 2010:2014, ni_eq = runif(5), dep_ME = runif(5, 0.01, 0.1), g_ME = runif(5)
pim_capital(DT)
```

---

read_firmdata_function

Read firm data

Description

Function used to generate the .RDS files of the MD datasets. It -selects NSI datafiles needed for MD datasets -determines which need to be imported in the envrionment -imports selected files and columns (using Rtool import_data.R) -performs the harmonization (using Rtool mdi_harmonize.R) -saves the MD datasets as .RDS

Usage

```
read_firmdata_function(related_md)
```

Arguments

related_md	MD dataset to harmonized and saved as .RDS

Value

harmonized .RDS file(s) of selected MD datasets

Author(s)

Cindy Jing Chen, 21/08 2020; Additions by EB, 18/01/2024; Mod. df 28/05/2024

Elena Sponga 10-09-2024

Reetuparna Vishwanath 13-09-2024

Index

conc_tab, 5
conc_type, 7
concord, 2
concord_nace, 3
concord_nace_complex, 4
consolidate_output, 8

disclosCrit, 8
disclose, 9

envir_prep, 10
estimate_markup, 11
estimate_outputelast, 12
estimate_prod, 13

gg_hist_density, 14
gg_scatter_reg, 15

harmonize_classification, 16
harmonize_variables, 17
hier_apply, 18

import_data, 19

log_warnings, 20

make_conc, 20
make_filelist, 21
md_cp2hier, 35
mdi_aggregate, 22
mdi_disagg, 23
mdi_export, 24
mdi_harmonize, 25
mdi_import_mpnames, 26
mdi_jointdist, 26
mdi_mergedatatables, 27
mdi_outlier, 28
mdi_reclass, 29
mdi_recode, 30
mdi_redefine, 31
mdi_regress, 31
mdi_relpath, 33
mdi_remap, 33
mdi_revalue, 34
mdi_sampleweights, 35

pim_capital, 36

read_firmdata_function, 37